

ARTI403 - Image Processing

Student Procedural Manual - Lab 1: Basics of Programming with Python

Name: Abdullah Alnaji

ID: 2240007157

Task #1: Exercise with Spyder

```
In [24]: # Quick warm-up exercise using Python/NumPy basics
import numpy as np

# simple list and loop practice
numbers = list(range(1, 11))
squares = [n**2 for n in numbers]
print("Numbers:", numbers)
print("Squares:", squares)

# simple NumPy array practice
arr = np.array(numbers)
print("Array:", arr)
print("Mean:", arr.mean(), "| Std:", arr.std())
```

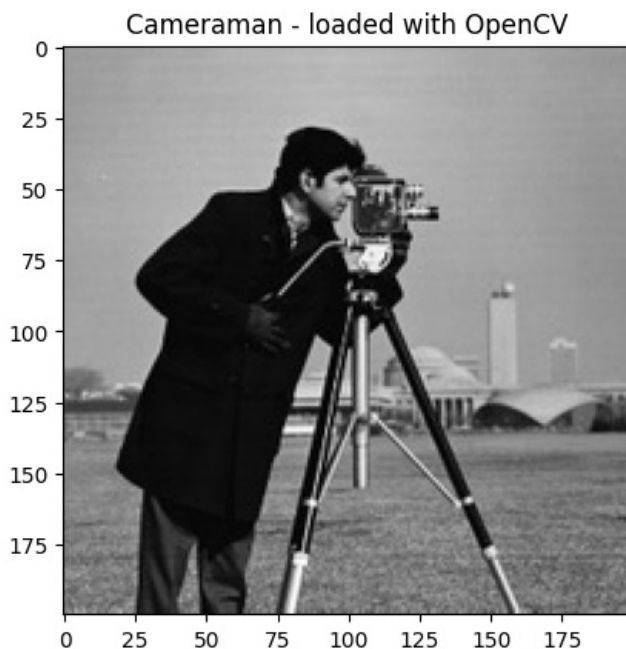
```
Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Array: [ 1  2  3  4  5  6  7  8  9 10]
Mean: 5.5 | Std: 2.8722813232690143
```

Task #2: Loading and Visualizing Images

a. Loading/reading an image using OpenCV

```
In [25]: import cv2
img = cv2.imread('cameraman.jpg') # Load the image data into a NumPy array

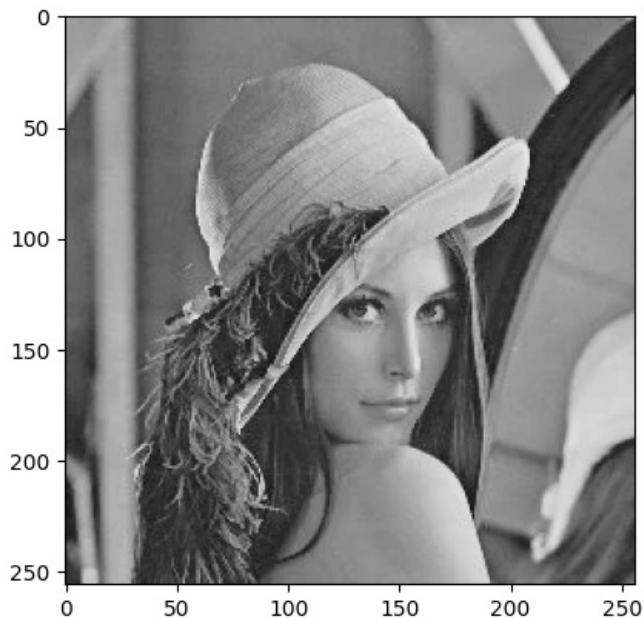
import matplotlib.pyplot as plt
plt.imshow(img)
plt.title('Cameraman - loaded with OpenCV')
plt.show()
```



b. Loading/reading image using PIL

```
In [26]: from PIL import Image
img2 = Image.open('lena_gray_256.webp') #load image into a PIL Image object

import matplotlib.pyplot as plt
import matplotlib.cm as cm
plt.imshow(img2, cmap = cm.Greys_r)
plt.show()
```



Task #3: Image Storing

a. Saving image to disk using OpenCV

```
In [27]: cv2.imwrite('new_image.jpg', img)
print("Saved new_image.jpg using OpenCV")
```

Saved new_image.jpg using OpenCV

b. Saving an image to disk using PIL

```
In [28]: img2.save('new_image2.jpg')
print("Saved new_image2.jpg using PIL")
```

Saved new_image2.jpg using PIL

Task #4: Display Image as an Array

```
In [29]: print(img.shape) # Note: img loaded using OpenCV is a NumPy array
print(img)

import numpy as np

# Convert the PIL image to a NumPy array
img_array = np.array(img2)

# Print the array
print(img_array.shape)
print(img_array)
```

```
(200, 200, 3)
[[[158 158 158]
  [158 158 158]
  [158 158 158]
  ...
  [154 154 154]
  [151 151 151]
  [155 155 155]]]
```

```

[[157 157 157]
 [158 158 158]
 [158 158 158]
 ...
 [154 154 154]
 [152 152 152]
 [153 153 153]]

[[157 157 157]
 [157 157 157]
 [157 157 157]
 ...
 [153 153 153]
 [153 153 153]
 [152 152 152]]

...

[[122 122 122]
 [123 123 123]
 [108 108 108]
 ...
 [121 121 121]
 [138 138 138]
 [120 120 120]]

[[120 120 120]
 [132 132 132]
 [130 130 130]
 ...
 [136 136 136]
 [137 137 137]
 [116 116 116]]

[[122 122 122]
 [125 125 125]
 [157 157 157]
 ...
 [124 124 124]
 [127 127 127]
 [119 119 119]]]
(256, 256, 3)
[[[162 162 162]
 [161 161 161]
 [158 158 158]
 ...
 [172 172 172]
 [164 164 164]
 [127 127 127]]

[[162 162 162]
 [161 161 161]
 [158 158 158]
 ...
 [175 175 175]
 [168 168 168]
 [127 127 127]]

[[162 162 162]
 [161 161 161]
 [158 158 158]
 ...
 [148 148 148]
 [121 121 121]
 [ 66 66 66]]

...

[[ 47 47 47]
 [ 51 51 51]
 [ 52 52 52]
 ...
 [ 85 85 85]
 [ 91 91 91]
 [ 90 90 90]]

[[ 42 42 42]
 [ 51 51 51]
 [ 52 52 52]

```

```
...
[ 95  95  95]
[ 99  99  99]
[ 97  97  97]]

[[ 38  38  38]
 [ 51  51  51]
 [ 52  52  52]
...
[102 102 102]
[111 111 111]
[111 111 111]]]
```

4. Assessment

Try performing different operations on `img_array` using NumPy, for example:

```
In [30]: # Example assessment operations on the array
print("Min pixel value:", img_array.min())
print("Max pixel value:", img_array.max())
print("Mean pixel value:", img_array.mean())

# Slice out the top-left 50x50 corner of the image
corner = img_array[:50, :50]
print("Corner shape:", corner.shape)

# Flip the image array horizontally
flipped = np.fliplr(img_array)
plt.imshow(flipped, cmap=cm.Greys_r)
plt.title('Flipped image (NumPy)')
plt.show()
```

```
Min pixel value: 1
Max pixel value: 255
Mean pixel value: 124.20447285970052
Corner shape: (50, 50, 3)
```

